Lecture 13: Basic Graph Algorithms

Michael Dinitz

October 15, 2026
601.433/633 Introduction to Algorithms

Introduction

Next 3-4 weeks: graphs!

- Super important abstractions, used all over the place in CS
- Most of my research is in graph algorithms (particularly when graph represents computer/communication network)
- Great course on Graph Theory in AMS

Today: review of basic graph algorithms from Data Structures, possibly one or two new

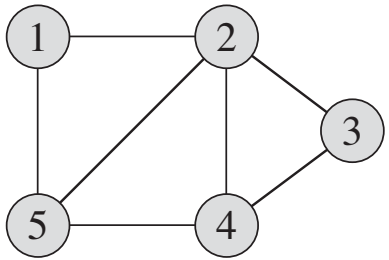
Basic Definitions

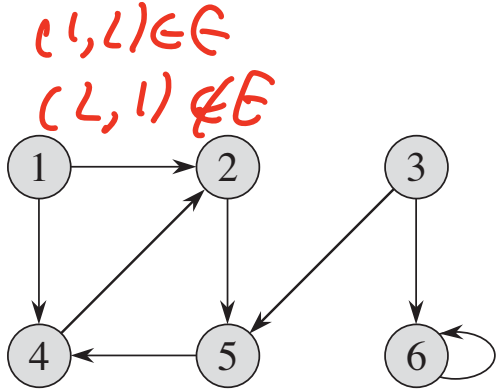
Definition

A graph $G = (V, E)$ is a pair where V is a set and $E \subseteq \binom{V}{2}$ (unordered pairs) or $E \subseteq V \times V$ (ordered pairs).

Notation:

- Elements of V are called *vertices* or *nodes*
- Elements of E are called *edges* or *arcs*.
- If $E \subseteq \binom{V}{2}$ then graph is *undirected*, if $E \subseteq V \times V$ graph is *directed*
- $|V| = n$ and $|E| = m$ (usually)
- So "size of input" $= n + m$





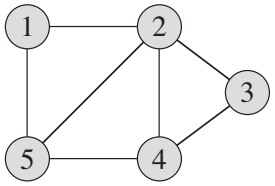
Representations

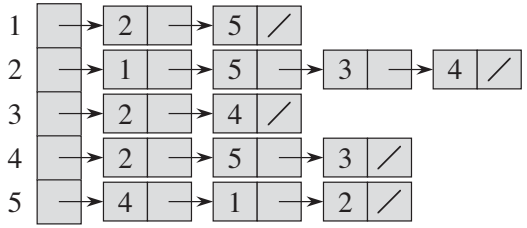
Adjacency List:

- Array A of length n
- $A[v]$ is linked list of vertices *adjacent* to v (edge from v to u)

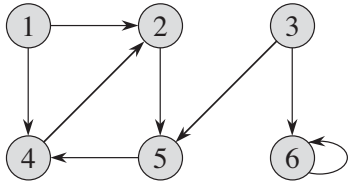
Adjacency Matrix:

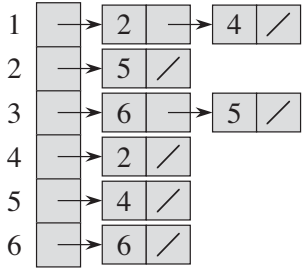
- $A \in \{0,1\}^{n \times n}$
- $A_{ij} = \begin{cases} 1 & \text{if } (i,j) \in E \\ 0 & \text{otherwise} \end{cases}$





	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0





	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

Representations (cont'd)

Adjacency List:

- Pros:

Representations (cont'd)

Adjacency List:

- Pros:
  - $O(n+m)$ space
  - Can iterate through edges adjacent to v very efficiently

Representations (cont'd)

Adjacency List:

- Pros:
  - $O(n+m)$ space
  - Can iterate through edges adjacent to v very efficiently
- Cons:

Representations (cont'd)

Adjacency List:

- Pros:
  - $O(n+m)$ space
  - Can iterate through edges adjacent to v very efficiently
- Cons:
  - Hard to check if an edge exists: $O(d(u))$ or $O(d(v))$ (where $d(v)$ is the degree of v: # edges with v as endpoint)

Representations (cont'd)

Adjacency List:

- Pros:
  - $O(n+m)$ space
  - Can iterate through edges adjacent to v very efficiently
- Cons:
  - Hard to check if an edge exists: $O(d(u))$ or $O(d(v))$ (where $d(v)$ is the degree of v: # edges with v as endpoint)

Adjacency Matrix:

- Pros:

Representations (cont'd)

Adjacency List:

- Pros:
  - $O(n+m)$ space
  - Can iterate through edges adjacent to v very efficiently
- Cons:
  - Hard to check if an edge exists: $O(d(u))$ or $O(d(v))$ (where $d(v)$ is the degree of v: # edges with v as endpoint)

Adjacency Matrix:

- Pros:
  - Can check if $e = (u, v)$ is an edge in $O(1)$ time

Representations (cont'd)

Adjacency List:

- Pros:
  - $O(n+m)$ space
  - Can iterate through edges adjacent to v very efficiently
- Cons:
  - Hard to check if an edge exists: $O(d(u))$ or $O(d(v))$ (where $d(v)$ is the degree of v: # edges with v as endpoint)

Adjacency Matrix:

- Pros:
  - Can check if $e = (u, v)$ is an edge in $O(1)$ time
- Cons:

Representations (cont'd)

Adjacency List:

- Pros:
  - $O(n+m)$ space
  - Can iterate through edges adjacent to v very efficiently
- Cons:
  - Hard to check if an edge exists: $O(d(u))$ or $O(d(v))$ (where $d(v)$ is the degree of v: # edges with v as endpoint)

Adjacency Matrix:

- Pros:
  - Can check if $e = (u, v)$ is an edge in $O(1)$ time
- Cons:
  - Takes $\Theta(n^2)$ space: if m small, lots wasted!
  - Iterating through edges incident on v takes time $\Theta(n)$, even if $d(v)$ small.

Representations (cont'd)

Adjacency List:

- Pros:
  - $O(n+m)$ space
  - Can iterate through edges adjacent to v very efficiently
- Cons:
  - Hard to check if an edge exists: $O(d(u))$ or $O(d(v))$ (where $d(v)$ is the degree of v: # edges with v as endpoint)

Adjacency Matrix:

- Pros:
  - Can check if $e = (u, v)$ is an edge in $O(1)$ time
- Cons:
  - Takes $\Theta(n^2)$ space: if m small, lots wasted!
  - Iterating through edges incident on v takes time $\Theta(n)$, even if $d(v)$ small.

This class: adjacency list unless otherwise specified.

Representations (cont'd)

Adjacency List:

- Pros:
  - $O(n+m)$ space
  - Can iterate through edges adjacent to v very efficiently
- Cons:
  - Hard to check if an edge exists: $O(d(u))$ or $O(d(v))$ (where $d(v)$ is the degree of v: # edges with v as endpoint)

Adjacency Matrix:

- Pros:
  - Can check if $e = (u, v)$ is an edge in $O(1)$ time
- Cons:
  - Takes $\Theta(n^2)$ space: if m small, lots wasted!
  - Iterating through edges incident on v takes time $\Theta(n)$, even if $d(v)$ small.

This class: adjacency list unless otherwise specified.

Any way to improve these?

Representations (cont'd)

Adjacency List:

- Pros:
  - $O(n+m)$ space
  - Can iterate through edges adjacent to v very efficiently
- Cons:
  - Hard to check if an edge exists: $O(d(u))$ or $O(d(v))$ (where $d(v)$ is the degree of v: # edges with v as endpoint)

Adjacency Matrix:

- Pros:
  - Can check if $e = (u, v)$ is an edge in $O(1)$ time
- Cons:
  - Takes $\Theta(n^2)$ space: if m small, lots wasted!
  - Iterating through edges incident on v takes time $\Theta(n)$, even if $d(v)$ small.

This class: adjacency list unless otherwise specified.

Any way to improve these?

- Replace adjacency *list* with adjacency *structure*: Red-black tree, hash table, etc.
- Not traditional, doesn't gain us much, and more complicated. But better!

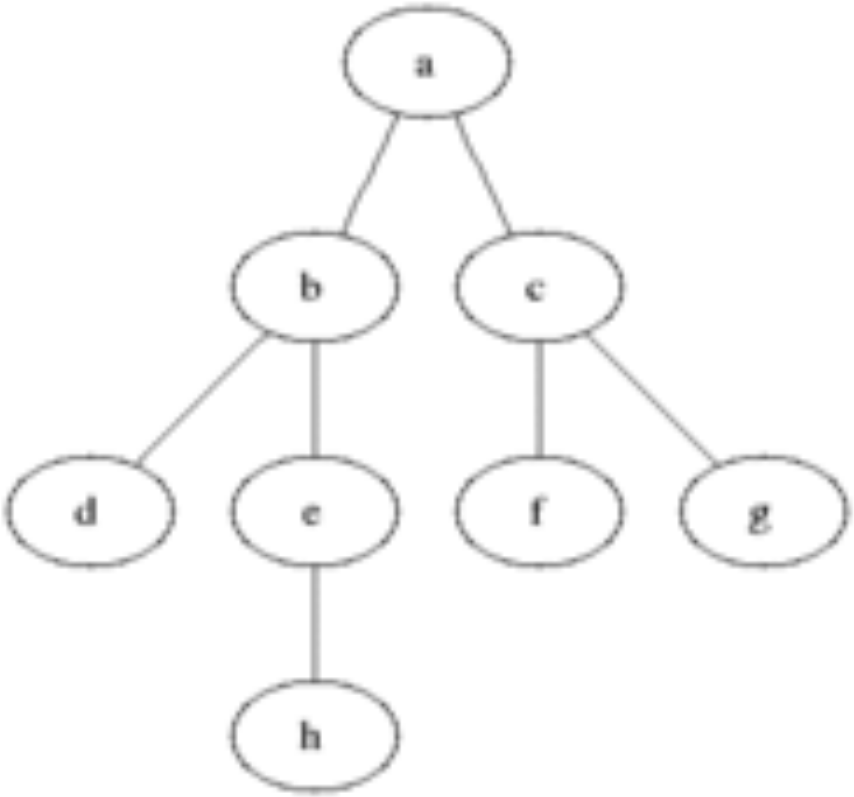
Breadth-First Search (BFS)

BFS Definition

Idea: explore graph in *levels* or *layers* from source s

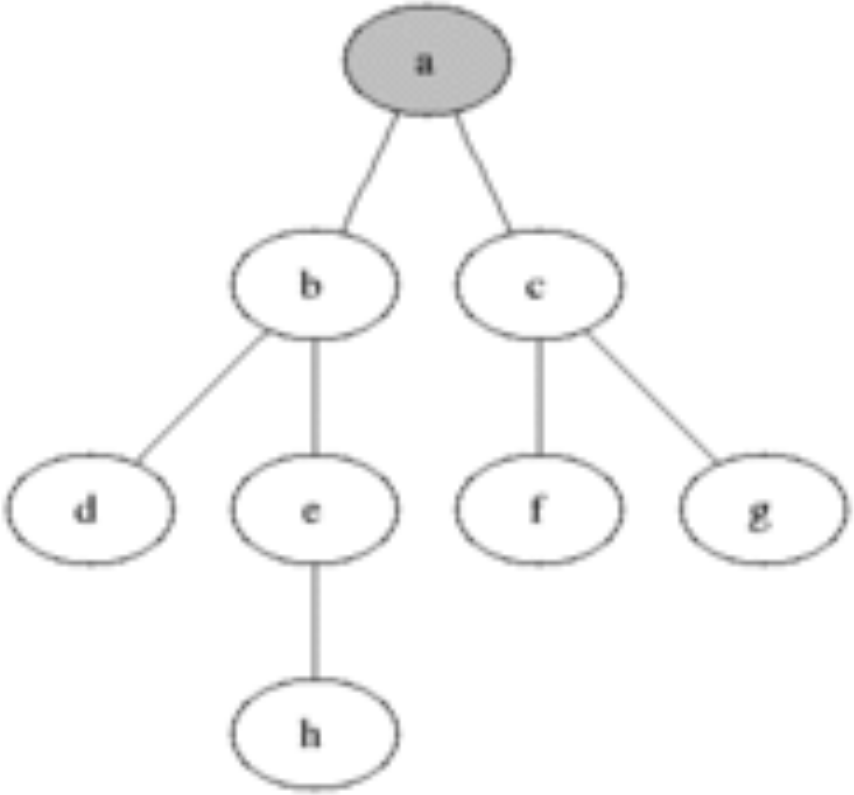
BFS Definition

Idea: explore graph in *levels* or *layers* from source ***s***



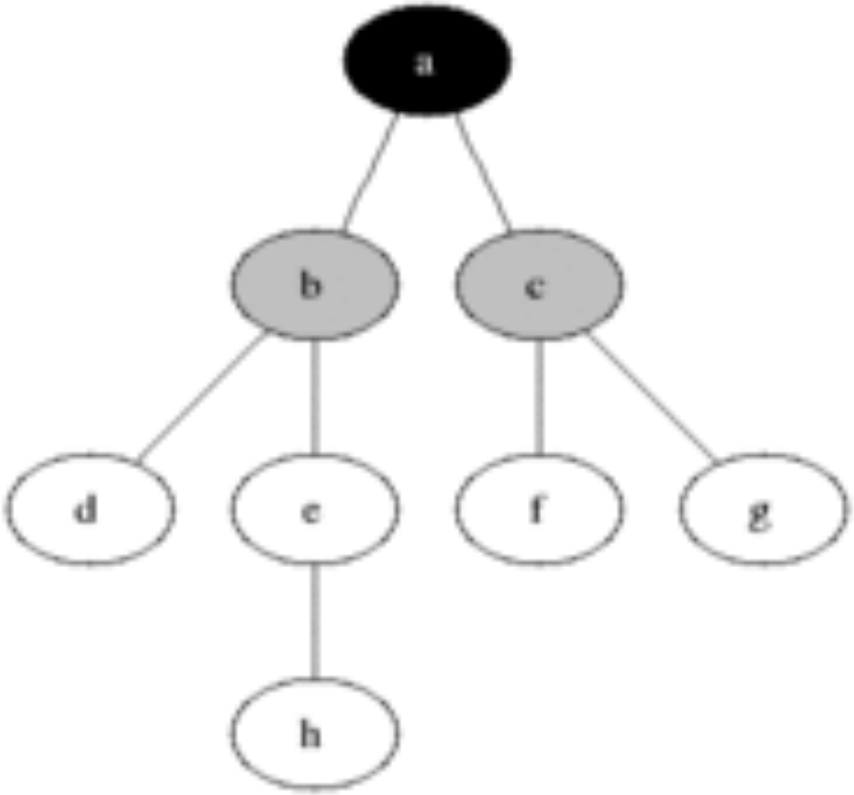
BFS Definition

Idea: explore graph in *levels* or *layers* from source ***s***



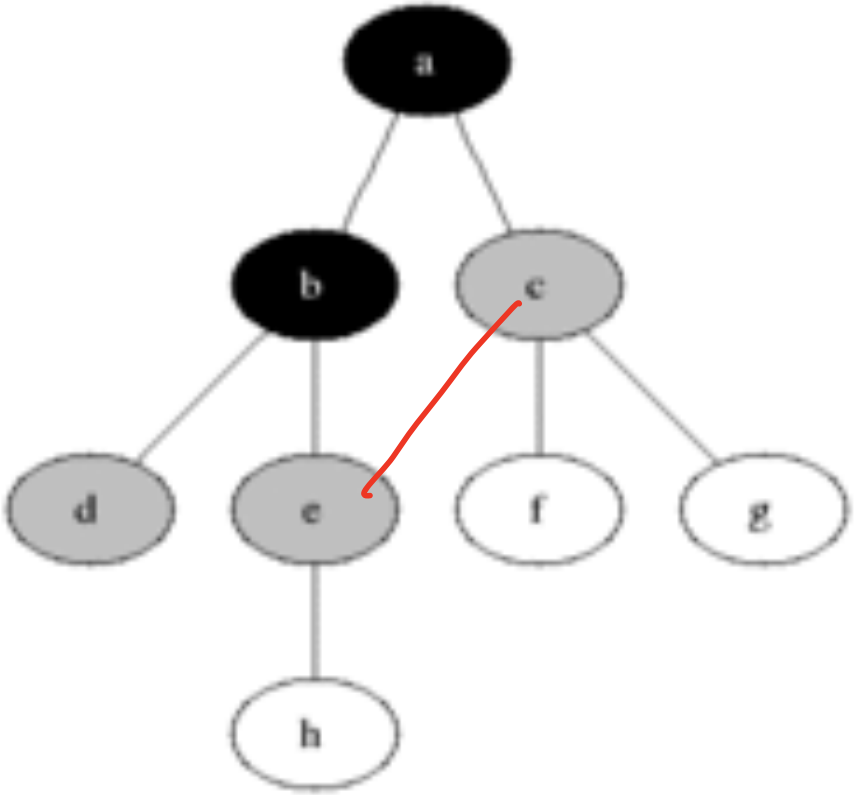
BFS Definition

Idea: explore graph in *levels* or *layers* from source ***s***



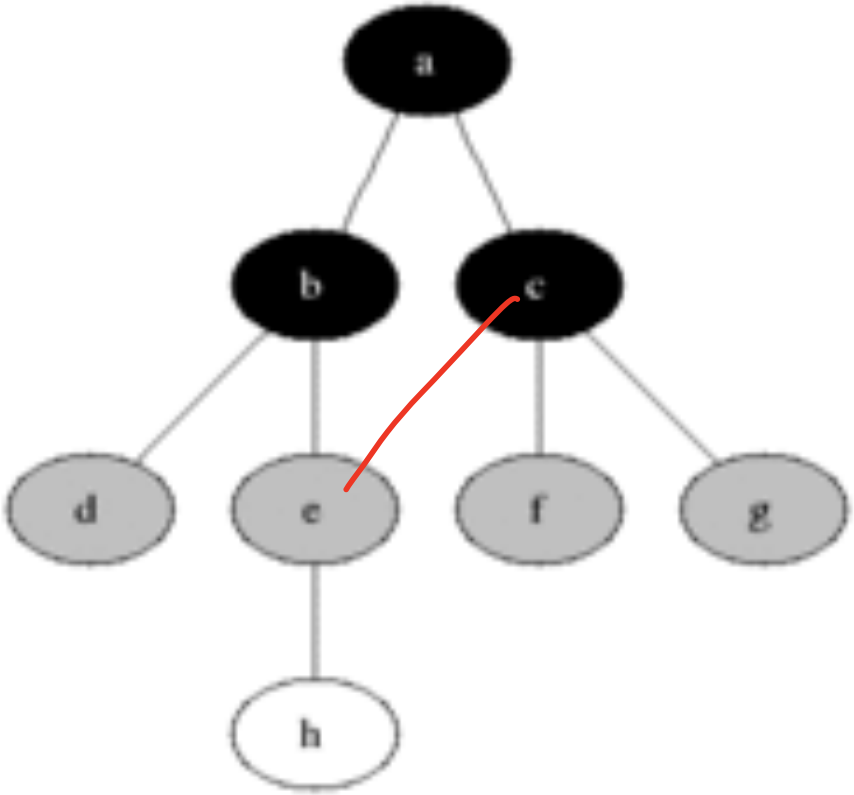
BFS Definition

Idea: explore graph in *levels* or *layers* from source ***s***



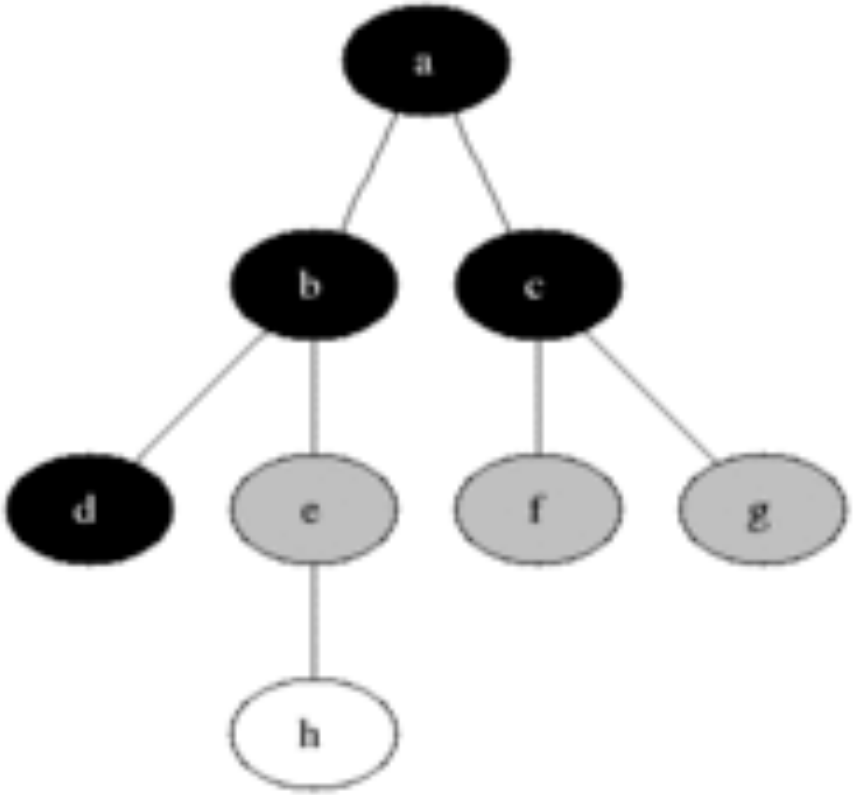
BFS Definition

Idea: explore graph in *levels* or *layers* from source ***s***



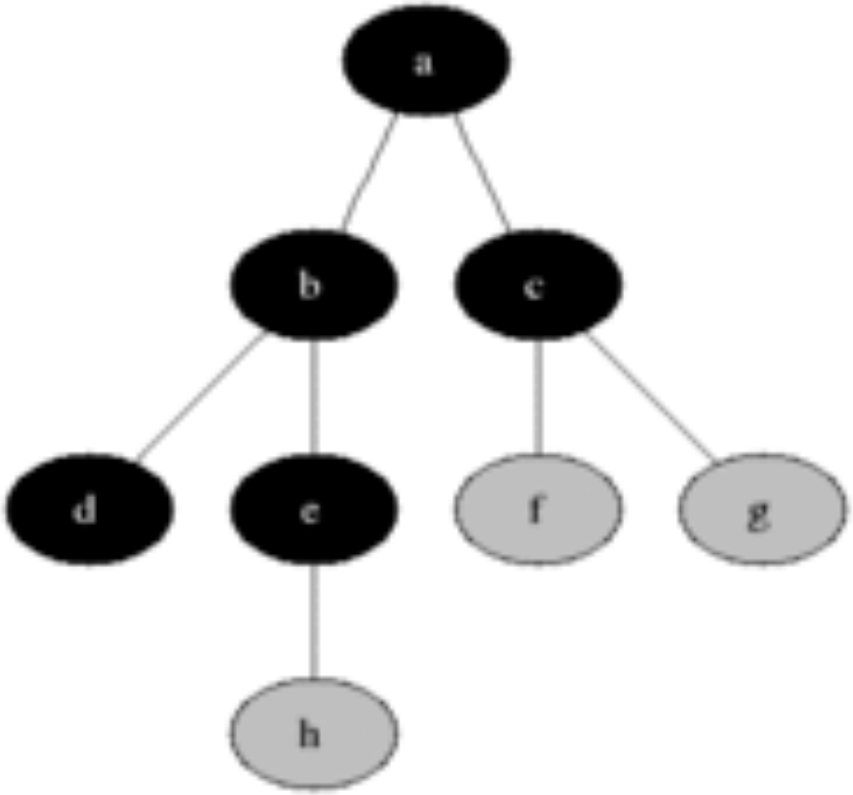
BFS Definition

Idea: explore graph in *levels* or *layers* from source ***s***



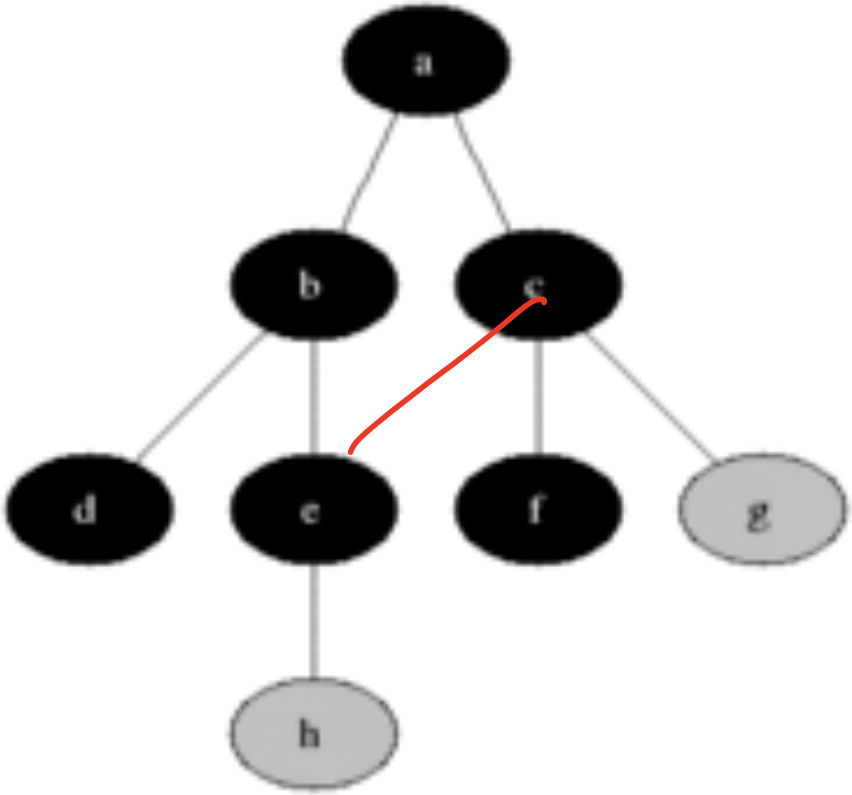
BFS Definition

Idea: explore graph in *levels* or *layers* from source ***s***



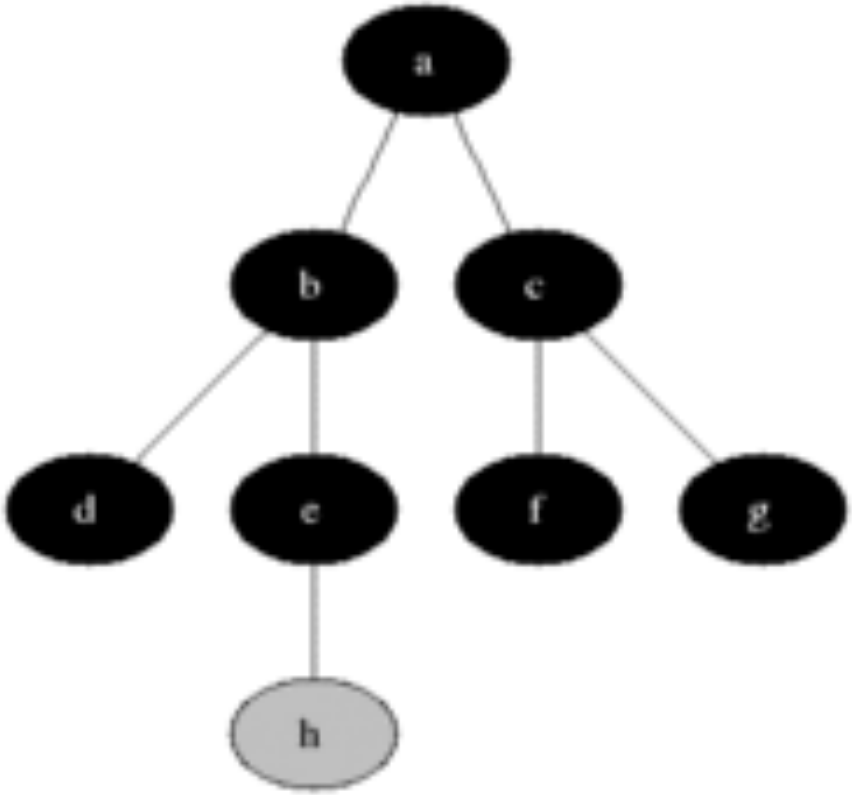
BFS Definition

Idea: explore graph in *levels* or *layers* from source ***s***



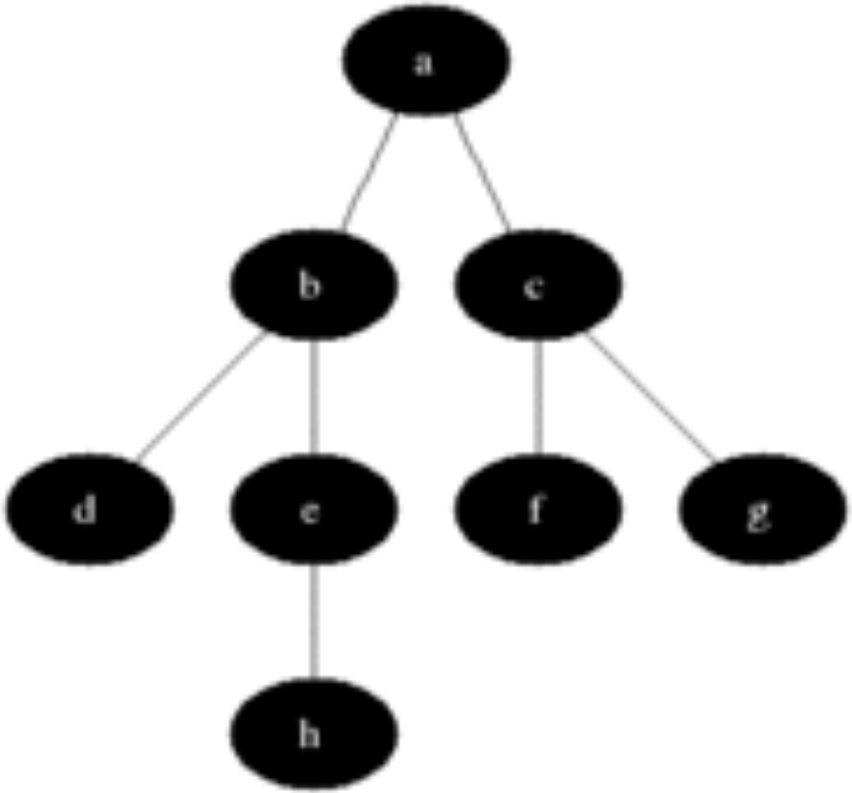
BFS Definition

Idea: explore graph in *levels* or *layers* from source ***s***



BFS Definition

Idea: explore graph in *levels* or *layers* from source ***s***



BFS Pseudocode

Idea: explore with a queue (FIFO)

```
BFS(G = (V, E), s) {
    Set mark(s) = True;
    Set mark(v) = False for all v ∈ V ∖ {s};
    Enqueue(s);
    while(queue not empty) {
        v = Dequeue();
        forall neighbors u of v {
            if(mark(u) == False) {
                mark(u) = True;
                Enqueue(u);
            }
        }
    }
}
```

BFS Pseudocode

Idea: explore with a queue (FIFO)

```
BFS(G = (V, E), s) {
    Set mark(s) = True;
    Set mark(v) = False for all v ∈ V ∖ {s};
    Enqueue(s);
    while(queue not empty) {
        v = Dequeue();
        forall neighbors u of v {
            if(mark(u) == False) {
                mark(u) = True;
                Enqueue(u);
            }
        }
    }
}
```

Running Time:

BFS Pseudocode

Idea: explore with a queue (FIFO)

```
BFS(G = (V, E), s) {
    Set mark(s) = True;
    Set mark(v) = False for all v ∈ V ∖ {s};
    Enqueue(s);
    while(queue not empty) {
        v = Dequeue();
        forall neighbors u of v {
            if(mark(u) == False) {
                mark(u) = True;
                Enqueue(u);
            }
        }
    }
}
```

Running Time: $O(n+m)$

BFS Pseudocode

Idea: explore with a queue (FIFO)

```
BFS(G = (V, E), s) {
    Set mark(s) = True;
    Set mark(v) = False for all v ∈ V ∖ {s};
    Enqueue(s);
    while(queue not empty) {
        v = Dequeue();
        forall neighbors u of v {
            if(mark(u) == False) {
                mark(u) = True;
                Enqueue(u);
            }
        }
    }
}
```

Running Time: $O(n+m)$

- $O(n)$ for initialization
- $O(m)$ for main while loop
  - Examine every edge twice: when each endpoint dequeued
  - Or (equivalent): Adjacency list scanned only when vertex dequeued

BFS Pseudocode

Idea: explore with a queue (FIFO)

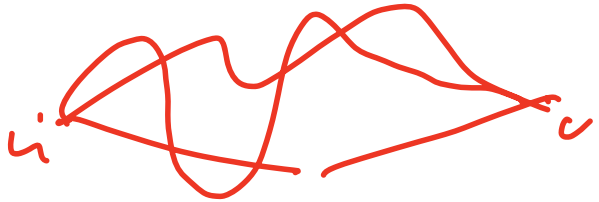
```
BFS(G = (V, E), s) {
    Set mark(s) = True;
    Set mark(v) = False for all v ∈ V ∖ {s};
    Enqueue(s);
    while(queue not empty) {
        v = Dequeue();
        forall neighbors u of v {
            if(mark(u) == False) {
                mark(u) = True;
                Enqueue(u);
            }
        }
    }
}
```

Running Time: $O(n+m)$

- $O(n)$ for initialization
- $O(m)$ for main while loop
  - Examine every edge twice: when each endpoint dequeued
  - Or (equivalent): Adjacency list scanned only when vertex dequeued

Note: edges that cause a node to be enqueued form a tree!

Correctness / Shortest Paths



Definition: Distance $d(u, v)$ from u to v is min # edges in any path from u to v

Theorem (informal): BFS(s) gives shortest paths from s to all other nodes

Correctness / Shortest Paths

Definition: Distance $d(u, v)$ from u to v is min # edges in any path from u to v

Theorem (informal): BFS(s) gives shortest paths from s to all other nodes

Proof Sketch: Let $L_i = \{v : d(s, v) = i\}$. Claim that layer i of BFS tree T equals L_i.

Correctness / Shortest Paths

Definition: Distance $d(u, v)$ from u to v is min # edges in any path from u to v

Theorem (informal): BFS(s) gives shortest paths from s to all other nodes

Proof Sketch: Let $L_i = \{v : d(s, v) = i\}$. Claim that layer i of BFS tree T equals L_i. Induction on i.

- Base case: $i = 0$.

Correctness / Shortest Paths

Definition: Distance $d(u, v)$ from u to v is min # edges in any path from u to v

Theorem (informal): BFS(s) gives shortest paths from s to all other nodes

Proof Sketch: Let $L_i = \{v : d(s, v) = i\}$. Claim that layer i of BFS tree T equals L_i. Induction on i.

- Base case: $i = 0$. ✓

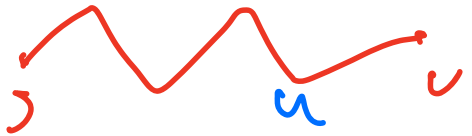
Correctness / Shortest Paths

Definition: Distance $d(u, v)$ from u to v is min # edges in any path from u to v

Theorem (informal): BFS(s) gives shortest paths from s to all other nodes

Proof Sketch: Let $L_i = \{v : d(s, v) = i\}$. Claim that layer i of BFS tree T equals L_i. Induction on i.

- Base case: $i = 0$. ✓
- Inductive step: consider $i > 0$, let $v \in L_i$.



Correctness / Shortest Paths

Definition: Distance $d(u, v)$ from u to v is min # edges in any path from u to v

Theorem (informal): BFS(s) gives shortest paths from s to all other nodes

Proof Sketch: Let $L_i = \{v : d(s, v) = i\}$. Claim that layer i of BFS tree T equals L_i. Induction on i.

- Base case: $i = 0$. ✓
- Inductive step: consider $i > 0$, let $v \in L_i$.
  Shortest $s - v$ path ends with edge $\{u, v\}$ with $u \in L_{i-1}$.

Correctness / Shortest Paths

Definition: Distance $d(u, v)$ from u to v is min # edges in any path from u to v

Theorem (informal): BFS(s) gives shortest paths from s to all other nodes

Proof Sketch: Let $L_i = \{v : d(s, v) = i\}$. Claim that layer i of BFS tree T equals L_i. Induction on i.

- Base case: $i = 0$. ✓
- Inductive step: consider $i > 0$, let $v \in L_i$.
  Shortest $s - v$ path ends with edge $\{u, v\}$ with $u \in L_{i-1}$.
  By induction, u in layer $i - 1$ of T

Correctness / Shortest Paths

Definition: Distance $d(u, v)$ from u to v is min # edges in any path from u to v

Theorem (informal): BFS(s) gives shortest paths from s to all other nodes

Proof Sketch: Let $L_i = \{v : d(s, v) = i\}$. Claim that layer i of BFS tree T equals L_i.
Induction on i.

- Base case: $i = 0$. ✓
- Inductive step: consider $i > 0$, let $v \in L_i$.
  Shortest $s - v$ path ends with edge $\{u, v\}$ with $u \in L_{i-1}$.
  By induction, u in layer $i - 1$ of T
  $\implies \{u, v\} \in T \implies v$ at layer i of T.

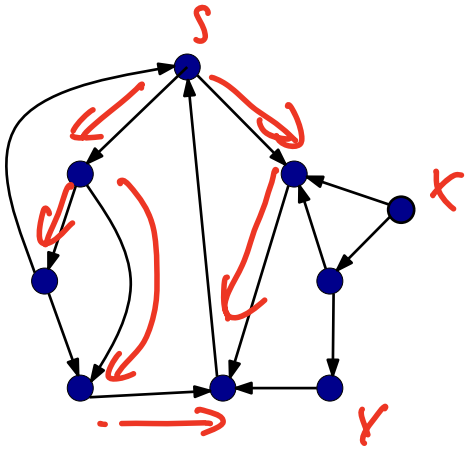
Depth-First Search (DFS)

DFS: Definition

Intuition: Instead of exploring wide (breadth), explore far (deep): just keep walking until see a node we've already seen, then backtrack!

```
Init: for each v ∈ V, mark(v) = False;

DFS(v) {
    mark(v) = True;
    for each edge (v, u) ∈ A[v] {
        if mark(u) == False then DFS(u);
    }
}
```

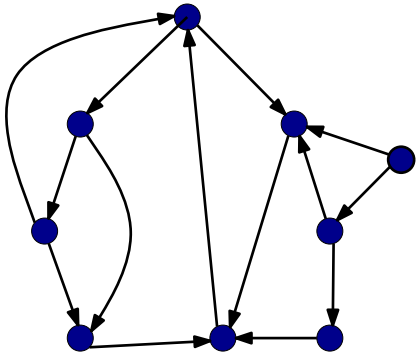


DFS: Definition

Intuition: Instead of exploring wide (breadth), explore far (deep): just keep walking until see a node we've already seen, then backtrack!

```
Init: for each v ∈ V, mark(v) = False;

DFS(v) {
    mark(v) = True;
    for each edge (v, u) ∈ A[v] {
        if mark(u) == False then DFS(u);
    }
}
```



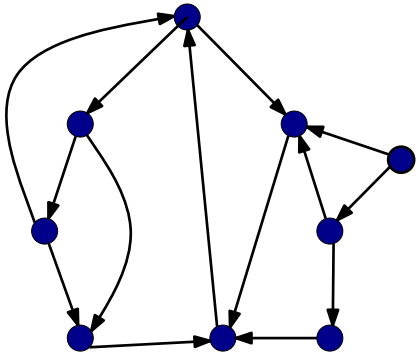
Running time:

DFS: Definition

Intuition: Instead of exploring wide (breadth), explore far (deep): just keep walking until see a node we've already seen, then backtrack!

```
Init: for each v ∈ V, mark(v) = False;

DFS(v) {
    mark(v) = True;
    for each edge (v, u) ∈ A[v] {
        if mark(u) == False then DFS(u);
    }
}
```



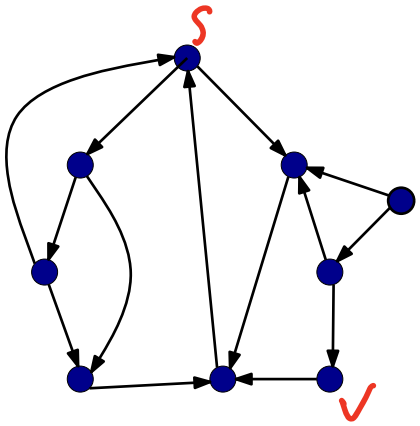
Running time: $O(m+n)$

DFS: Definition

Intuition: Instead of exploring wide (breadth), explore far (deep): just keep walking until see a node we've already seen, then backtrack!

```
Init: for each v ∈ V, mark(v) = False;

DFS(v) {
    mark(v) = True;
    for each edge (v, u) ∈ A[v] {
        if mark(u) == False then DFS(u);
    }
}
```



Running time: $O(m+n)$

- $O(n)$ initialization
- Every edge considered at most twice

DFS: Correctness

Definition: u is *reachable* from v if there is a path $v = v_0, v_1, \ldots, v_k = u$ such that $(v_i, v_{i+1}) \in E$ for all $i \in \{0, 1, \ldots, k-1\}$.

Theorem

When DFS(v) terminates, it has visited (marked) all nodes that are reachable from v.

Proof.

Suppose u reachable from v but not marked when DFS(v) terminates.

DFS: Correctness

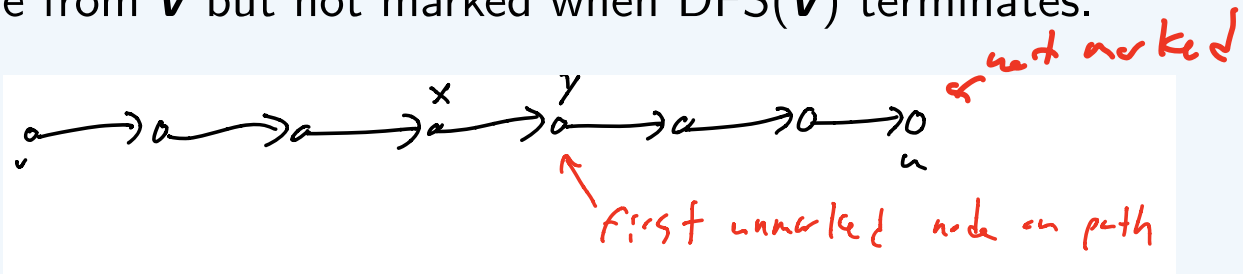
Definition: u is *reachable* from v if there is a path $v = v_0, v_1, \ldots, v_k = u$ such that $(v_i, v_{i+1}) \in E$ for all $i \in \{0, 1, \ldots, k-1\}$.

Theorem

When DFS(v) terminates, it has visited (marked) all nodes that are reachable from v.

Proof.

Suppose u reachable from v but not marked when DFS(v) terminates.



DFS: Correctness

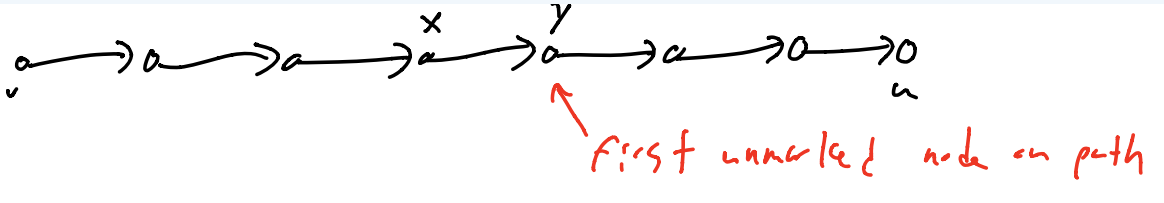
Definition: u is *reachable* from v if there is a path $v = v_0, v_1, \ldots, v_k = u$ such that $(v_i, v_{i+1}) \in E$ for all $i \in \{0, 1, \ldots, k-1\}$.

Theorem

When DFS(v) terminates, it has visited (marked) all nodes that are reachable from v.

Proof.

Suppose u reachable from v but not marked when DFS(v) terminates.



x is marked so DFS(x) must have been called

DFS: Correctness

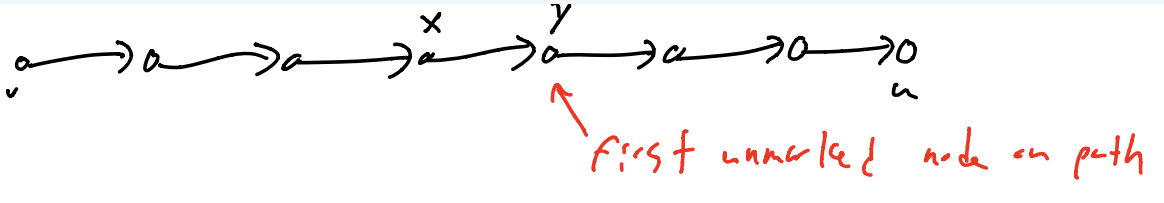
Definition: u is *reachable* from v if there is a path $v = v_0, v_1, \ldots, v_k = u$ such that $(v_i, v_{i+1}) \in E$ for all $i \in \{0, 1, \ldots, k-1\}$.

Theorem

When DFS(v) terminates, it has visited (marked) all nodes that are reachable from v.

Proof.

Suppose u reachable from v but not marked when DFS(v) terminates.



x is marked so DFS(x) must have been called

$\Longrightarrow$ y was either marked or DFS(y) called and it became marked.

DFS: Correctness

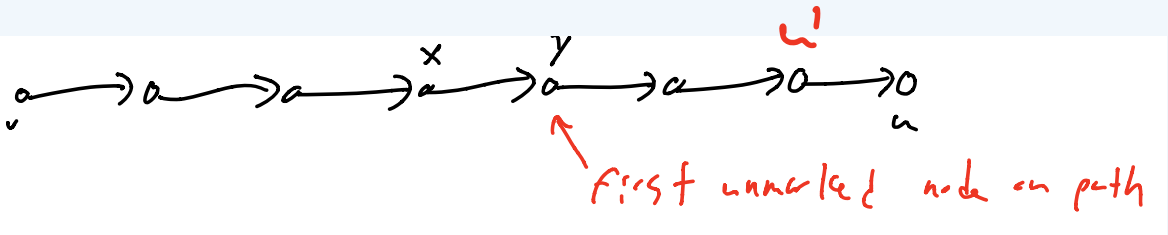
Definition: u is *reachable* from v if there is a path $v = v_0, v_1, \ldots, v_k = u$ such that $(v_i, v_{i+1}) \in E$ for all $i \in \{0, 1, \ldots, k-1\}$.

Theorem

When DFS(v) terminates, it has visited (marked) all nodes that are reachable from v.

Proof.

Suppose u reachable from v but not marked when DFS(v) terminates.



x is marked so DFS(x) must have been called

$\Longrightarrow$ y was either marked or DFS(y) called and it became marked.

Contradiction. □

Graph variant

After DFS($\boldsymbol{v}$), node marked if and only if reachable from $\boldsymbol{v}$.

Might want to continue until all nodes marked.

```
DFS(G) {
    for all v ∈ V, set mark(v) = False;
    while there exists an unmarked node v {
        DFS(v);
    }
}
```

Timestamps

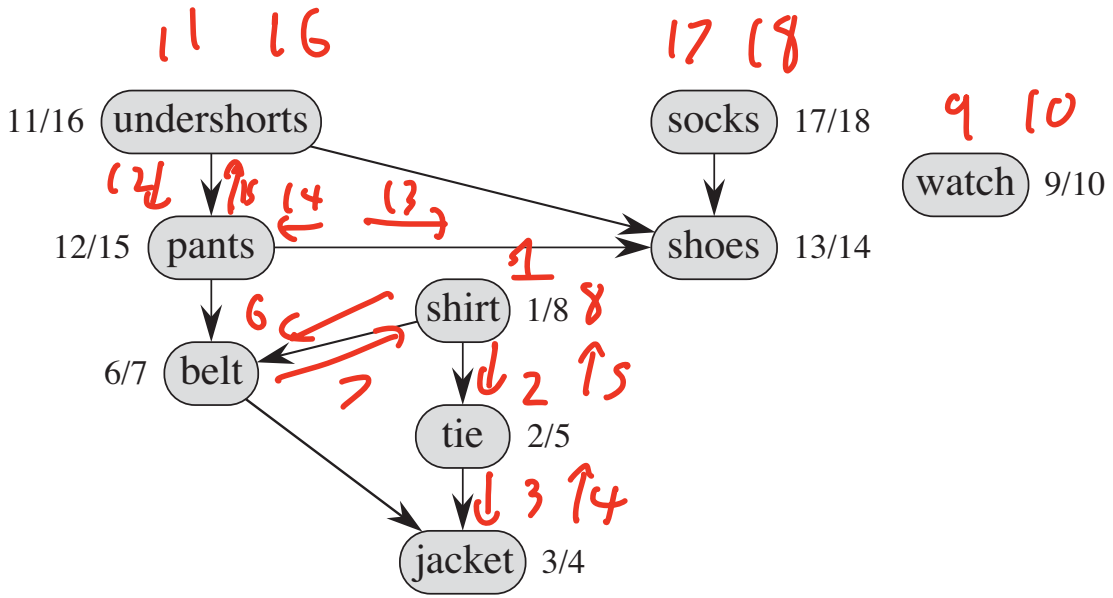
Explicitly keep track of "start" and "finishing" times

- Replaces ***mark***

```
DFS(G) {
    t = 0;
    for all v ∈ V {
        start(v) = 0;
        finish(v) = 0;
    }
    while ∃v ∈ V with start(v) = 0 {
        DFS(v);
    }
}
```

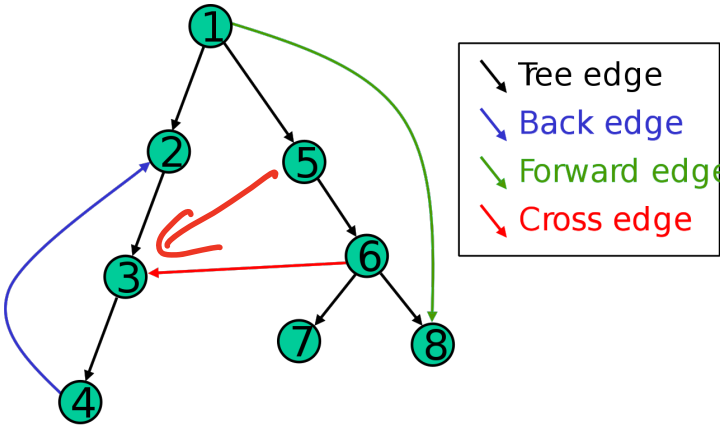
```
DFS(v) {
    t = t + 1;
    start(v) = t;
    for each edge (v, u) ∈ A[v] {
        if start(u) == 0 then DFS(u);
    }
    t = t + 1;
    finish(v) = t;
}
```

Timestamp Example



Edge Types

DFS naturally gives a spanning forest: edge (v, u) if DFS(v) calls DFS(u)



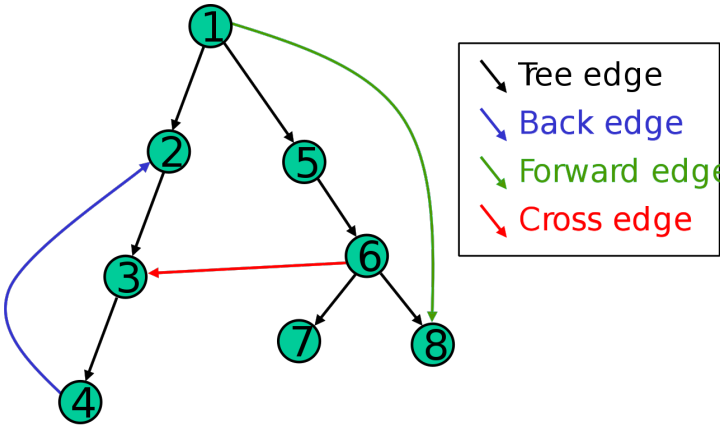
Forward Edges: (v, u) such that u descendant of v (includes tree edges)

Back Edges: (v, u) such that u an ancestor of v

Cross Edges: (v, u) such that u neither a descendant nor an ancestor of v

Edge Types

DFS naturally gives a spanning forest: edge (v, u) if DFS(v) calls DFS(u)



Forward Edges: (v, u) such that u descendant of v (includes tree edges)

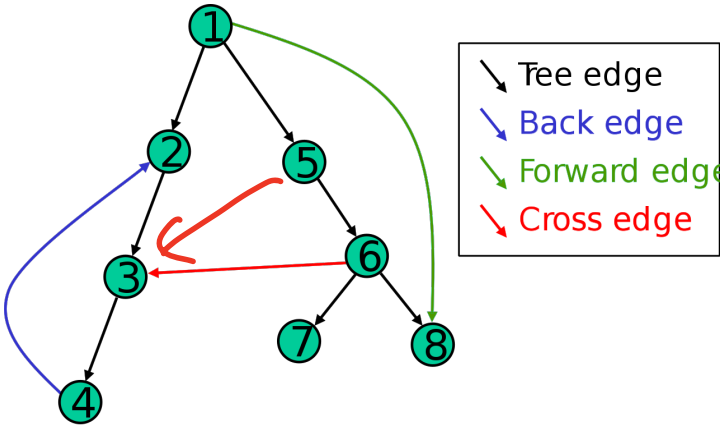
$start(v) < start(u) < finish(u) < finish(v)$

Back Edges: (v, u) such that u an ancestor of v

Cross Edges: (v, u) such that u neither a descendant nor an ancestor of v

Edge Types

DFS naturally gives a spanning forest: edge (v, u) if DFS(v) calls DFS(u)



Forward Edges: (v, u) such that u descendant of v (includes tree edges)

$start(v) < start(u) < finish(u) < finish(v)$

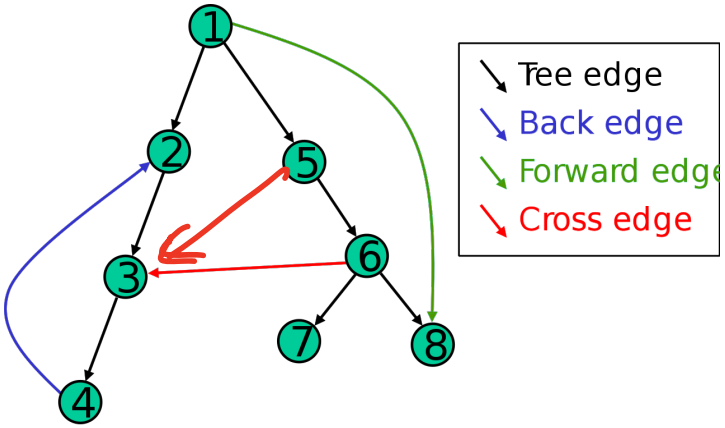
Back Edges: (v, u) such that u an ancestor of v

$start(u) < start(v) < finish(v) < finish(u)$

Cross Edges: (v, u) such that u neither a descendant nor an ancestor of v

Edge Types

DFS naturally gives a spanning forest: edge (v, u) if DFS(v) calls DFS(u)



Forward Edges: (v, u) such that u descendant of v (includes tree edges)

$start(v) < start(u) < finish(u) < finish(v)$

Back Edges: (v, u) such that u an ancestor of v

$start(u) < start(v) < finish(v) < finish(u)$

Cross Edges: (v, u) such that u neither a descendant nor an ancestor of v

$start(u) < finish(u) < start(v) < finish(v)$